CODE:

```
#include <bits/stdc++.h>
#include <chrono>
using namespace std;
using namespace std::chrono;
// GRAPH REPRESENTATION: ADJACENCY LIST
class GraphList {
public:
    int V;
    vector<vector<int>> adj;
    GraphList(int V) {
        this->V = V;
        adj.resize(V);
    }
    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    void BFS(int start) {
        vector<bool> visited(V, false);
        queue<int> q;
        visited[start] = true;
        q.push(start);
        while (!q.empty()) {
            int node = q.front();
            q.pop();
            for (int neighbor : adj[node]) {
```

```
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
}
void DFSUtil(int node, vector<bool>& visited) {
    visited[node] = true;
    for (int neighbor : adj[node])
        if (!visited[neighbor])
            DFSUtil(neighbor, visited);
}
void DFS(int start) {
    vector<bool> visited(V, false);
    DFSUtil(start, visited);
}
};
/ GRAPH REPRESENTATION: COMPRESSED SPARSE ROW (CSR)
class GraphCSR {
public:
    int V;
    vector<int> row_ptr;
    vector<int> col_idx;
    GraphCSR(int V, const vector<pair<int, int>>& edges) {
        this->V = V;
        row_ptr.resize(V + 1, 0);
```

```
    for (auto& e : edges)
        row_ptr[e.first + 1]++;
    for (int i = 1; i <= V; ++i)
        row_ptr[i] += row_ptr[i - 1];
    col_idx.resize(edges.size() * 2);
    vector<int> temp = row_ptr;
    for (auto& e : edges) {
        col_idx[temp[e.first]++] = e.second;
        col_idx[temp[e.second]++] = e.first;
    }
}
void BFS(int start) {
    vector<bool> visited(V, false);
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty()) {
        int node = q.front();
        q.pop();
        for (int i = row_ptr[node]; i < row_ptr[node + 1]; ++i) {
            int neighbor = col_idx[i];
            if (!visited[neighbor]) {
                visited[neighbor] = true;
                q.push(neighbor);
            }
        }
    }
```

```
    }
    void DFSUtil(int node, vector<bool>& visited) {
        visited[node] = true;
        for (int i = row_ptr[node]; i < row_ptr[node + 1]; ++i) {
            int neighbor = col_idx[i];
            if (!visited[neighbor])
                DFSUtil(neighbor, visited);
        }
    }
    void DFS(int start) {
        vector<bool> visited(V, false);
        DFSUtil(start, visited);
    }
};
// PERFORMANCE MEASUREMENT
template <typename F>
double measure(F func) {
    auto start = high_resolution_clock::now();
    func();
    auto end = high_resolution_clock::now();
    return duration<double, milli>(end - start).count();
}
int main() {
    srand(time(0));
    vector<int> sizes = {1000, 5000, 10000};
    cout << "\nComparative Analysis of BFS and DFS\n";
    cout << "-------------------------------------\n";
```

```
    cout << "Vertices\tBFS(List)\tDFS(List)\tBFS(CSR)\tDFS(CSR)\n";
    for (int V : sizes) {
        vector<pair<int, int>> edges;
        for (int i = 0; i < V * 3; ++i) {
            int u = rand() % V;
            int v = rand() % V;
            if (u != v)
                edges.push_back({u, v});
        }
        GraphList g1(V);
        for (auto& e : edges)
            g1.addEdge(e.first, e.second);
        GraphCSR g2(V, edges);
        double bfs_list = measure([&]() { g1.BFS(0); });
        double dfs_list = measure([&]() { g1.DFS(0); });
        double bfs_csr = measure([&]() { g2.BFS(0); });
        double dfs_csr = measure([&]() { g2.DFS(0); });
        cout << V << "\t\t" << bfs_list << "\t\t" << dfs_list
             << "\t\t" << bfs_csr << "\t\t" << dfs_csr << endl;
    }
    return 0;
}
```

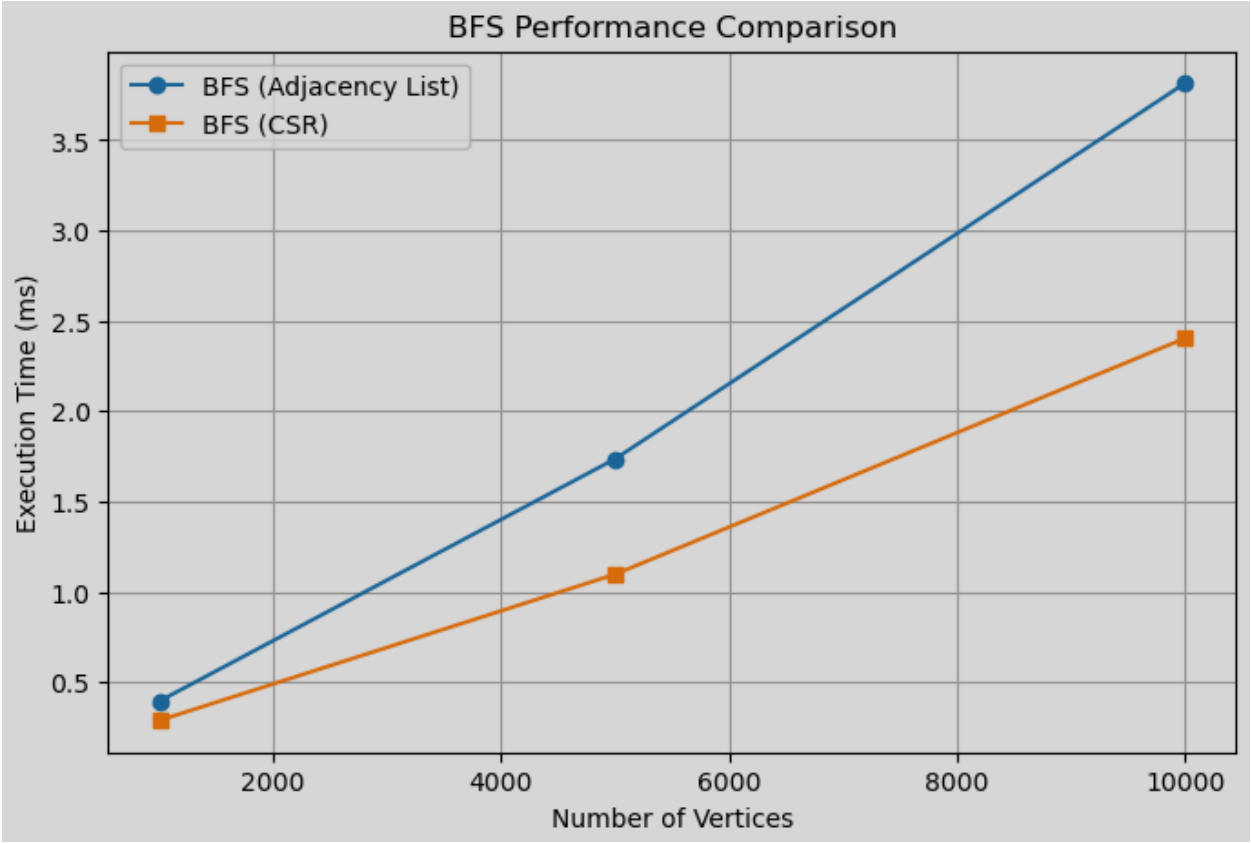
OUTPUT:

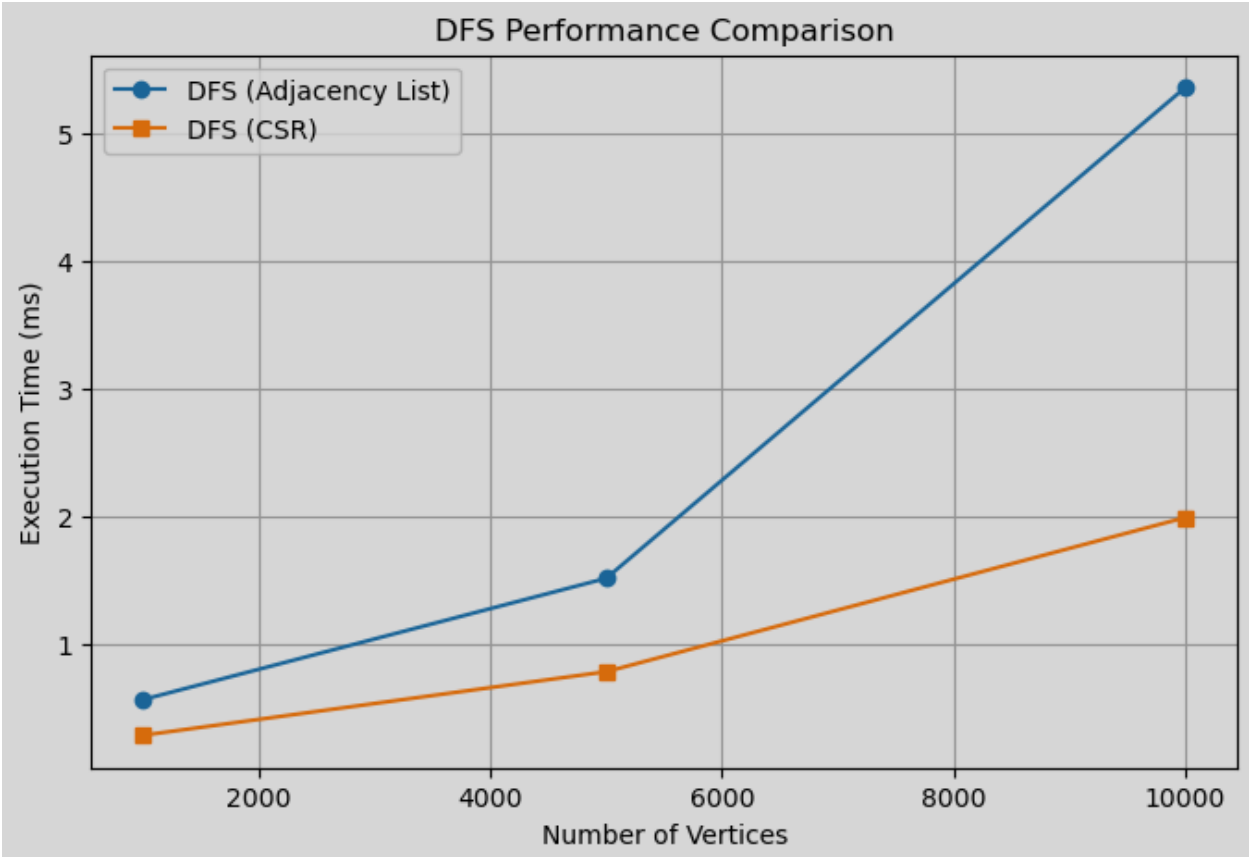
```
G:\.VIT\Courses\DSA\DSA_PR

Comparative Analysis of BFS and DFS
-----------------------------------
Vertices        BFS(List)       DFS(List)       BFS(CSR)        DFS(CSR)
1000            0.3962          0.5592          0.2891          0.2791
5000            1.7368          1.5117          1.0982          0.7782
10000           3.8149          5.3644          2.4037          1.9884

Process returned 0 (0x0)   execution time : 0.057 s
Press any key to continue.
```

Comparison Plot:

```
import matplotlib.pyplot as plt

# Data

vertices = [1000, 5000, 10000]

bfs_list = [0.3962, 1.7368, 3.8149]

dfs_list = [0.5592, 1.5117, 5.3644]

bfs_csr = [0.2891, 1.0982, 2.4037]

dfs_csr = [0.2791, 0.7782, 1.9884]

# BFS comparison

plt.figure(figsize=(8,5))

plt.plot(vertices, bfs_list, 'o-', label='BFS (Adjacency List)')

plt.plot(vertices, bfs_csr, 's-', label='BFS (CSR)')

plt.xlabel('Number of Vertices')

plt.ylabel('Execution Time (ms)')

plt.title('BFS Performance Comparison')

plt.legend()

plt.grid(True)

plt.show()
```

```
# DFS comparison
plt.figure(figsize=(8,5))
plt.plot(vertices, dfs_list, 'o-', label='DFS (Adjacency List)')
plt.plot(vertices, dfs_csr, 's-', label='DFS (CSR)')
plt.xlabel('Number of Vertices')
plt.ylabel('Execution Time (ms)')
plt.title('DFS Performance Comparison')
plt.legend()
plt.grid(True)
plt.show()
```




DFS Performance Comparison
DFS (Adjacency List)
DFS (CSR)
Execution Time (ms)
1
2
3
4
5
2000
4000
6000
8000
10000
Number of Vertices